

MODULE 5: SOFTWARE QUALITY

13.1 INTRODUCTION

Quality:

- Quality is generally agreed to be ‘ **a good thing**’ .
- In a practice what people really mean by the ‘quality’ of a system can be vague, undefined attribute.
- We need to define precisely what qualities we require of a system.

Objective assessment:

- We need to judge objectively whether a system meets our quality requirements and this need measurement.
- Critical for package selection, e.g., Brigitte at Brightmouth College.

Now days, delivering a high-quality product is one of the major objectives of all organizations. Traditionally, the quality of a product means that how much it gets fit into its specified purpose. A product is of *good quality if it performs according to the user's requirements*. Good quality software should meet all objectives defined in the SRS document. It is the responsibility of the quality managers to ensure that the software attains a required level of quality.

Waiting until the system is complete to measure quality is too late.

During development, it's important to:

- Assess the likely quality of the final system.
- Ensure development methods will produce the desired quality.

Potential customers, like Amanda at IOE, might focus on:

- Checking if suppliers use the best development methods.
- Ensuring these methods will lead to the required quality in the final system.

13.2 THE PLACE OF SOFTWARE QUALITY IN PROJECT PLANNING

Quality will be of concern at all stages of project planning and execution, but will be particular interest at Stepwise framework.

Step 1 : Identifying project scope and objectives Some objective could relate to the quality of the application to be delivered.

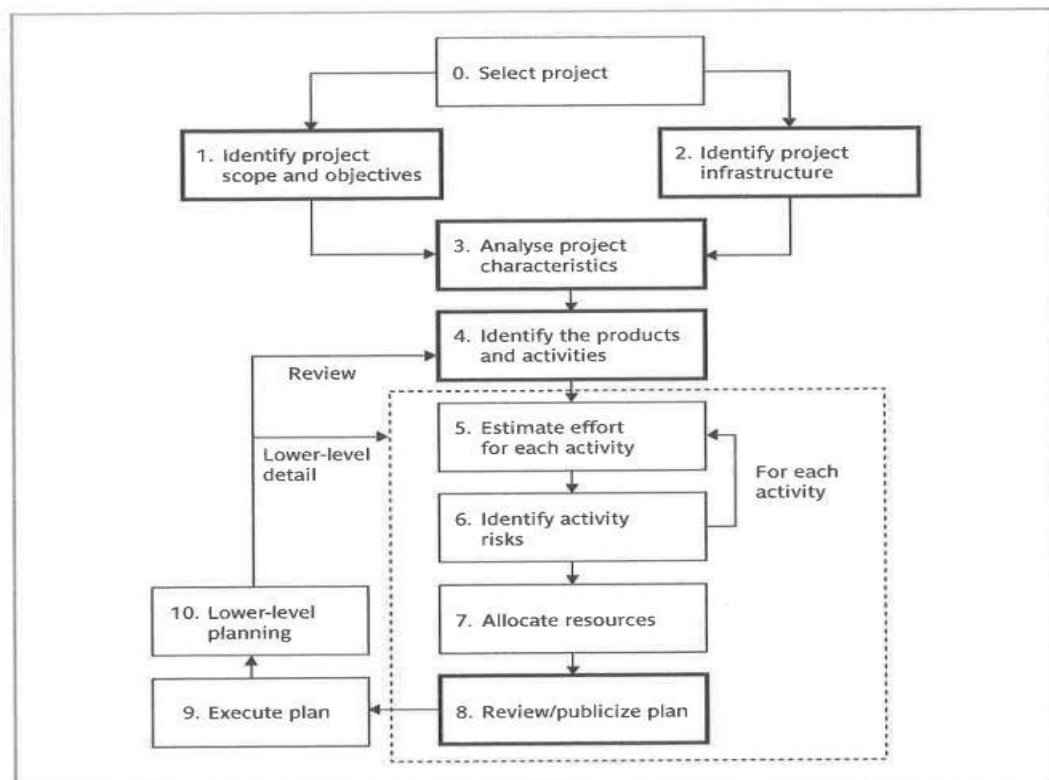
Step 2 : Identifying project infrastructure Within this step activity 2.2 involves identifying installation standards and procedures. Some of these will almost certainly be about quality requirements.

Step 3 : Analyze project characteristics. In this activity the application to be implemented will be examined to see if it has any special quality requirements.

Example: Safety-critical applications might require additional activities such as n-version development, where multiple teams develop versions of the same software to cross-check outputs.

Step 4 : Identify the product and activities of the project. It is at that point the entry, exit and process requirement are identified for each activity.

Step 8: Review and publicize plan. At this stage the overall quality aspects of the project plan are reviewed.



13.3 THE IMPORTANCE OF SOFTWARE QUALITY

Now a days, **quality is the important aspect of all organization**. Good quality software is the requirement of all users. There are so many reasons that describe why the quality of software is important; few among of those which are most important are described below:

Increasingly criticality of software:

- The final customer or user is naturally anxious about the general quality of software especially about the reliability.
- They are concern about the safety because of their dependency on the software system such as aircraft control system are more safety critical systems.

Earlier detection of errors during development-

- As software is developed through a number of phases; output of one phase is given as input to the other one. So, if error in the initial phase is not found, then at the later stage, it is difficult to fix that error and also the cost indulged is more.

The intangibility of software :

- Difficulty in verifying the satisfactory completion of project tasks.
- Tangibility is achieved by requiring developers to produce "deliverables" that can be examined for quality.

Accumulating errors during software development :

- Errors in earlier steps can propagate and accumulate in later steps.
- Errors found later in the project are more expensive to fix.
- The unknown number of errors makes the debugging phase difficult to control.

13.4 DEFINING SOFTWARE QUALITY

Quality is a rather vague term and we need to define carefully what we mean by it. For any software system there should be three specifications.

- **A functional specification** describing what the system is to do
- **A quality specification** concerned with how well the function are to operate
- **A resource specification** concerned with how much is to be spent on the system.

Attempt to identify specific product qualities that are appropriate to software , for instance, grouped software qualities into three sets. Product operation qualities, Product revision qualities and product transition qualities.

Product operation qualities:

Correctness: The extent to which a program satisfies its specification and fulfil user objective.

Reliability: the extent to which a program can be expected to perform its intended function with required precision.

Efficiency: The amounts of computer resource required by software.

Integrity: The extent to which access to software or data by unauthorized persons can be controlled.

Usability: The effort required to learn, operate, prepare input and interprets output.

Product Revision Qualities:

Maintainability: The effort required to locate and fix an error in an operational program

Testability: The effort required to test a program to ensure it performs its intended function.

Flexibility: The effort required to modify an operational program.

Product Transition Qualities:

Portability: The efforts require to transfer a program from one hardware configuration and or software system environment to another.

Reusability: The extent to which a program can be used in other applications.

Interoperability: The efforts required to couple one system to another.

When there is concerned about the need for a specific quality characteristic in a software product then a quality specification with the following minimum details should be drafted .

1. Definition/Description

Definition: Clear definition of the quality characteristic.

Description: Detailed description of what the quality characteristic entails.

2. Scale o Unit of Measurement:

The unit used to measure the quality characteristic (e.g., faults per thousand lines of code).

3. Test

Practical Test: The method or process used to test the extent to which the quality attribute exists.

4. Minimally Acceptable

Worst Acceptable Value: The lowest acceptable value, below which the product would be rejected.

5. Target Range

Planned Range: The range of values within which it is planned that the quality measurement value should lie.

6. Current Value

Now: The value that applies currently to the quality characteristic.

Measurements Applicable to Quality Characteristics in Software

When assessing quality characteristics in software, multiple measurements may be applicable. For example, in the case of reliability, measurements could include:

1. Availability:

Definition: Percentage of a particular time interval that a system is usable.

Scale: Percentage (%).

Test: Measure the system's uptime versus downtime over a specified period.

Minimally Acceptable: Typically, high availability is desirable; specifics depend on system requirements.

Target Range: E.g., 99.9% uptime.

2. Mean Time Between Failures (MTBF):

Definition: Total service time divided by the number of failures.

Scale: Time (e.g., hours, days).

Test: Calculate the average time elapsed between system failures.

Minimally Acceptable: Longer MTBF indicates higher reliability; minimum varies by system criticality.

Target Range: E.g., MTBF of 10,000 hours.

3. Failure on Demand:

Definition: Probability that a system will not be available when required, or probability that a transaction will fail.

Scale: Probability (0 to 1).

Test: Evaluate the system's response to demand or transaction processing.

Minimally Acceptable: Lower probability of failure is desired; varies by system criticality.

Target Range: E.g., Failure on demand probability of less than 0.01.

4. Support Activity:

Definition: Number of fault reports generated and processed.

Scale: Count (number of reports).

Test: Track and analyze the volume and resolution time of fault reports.

Minimally Acceptable: Lower number of fault reports indicates better reliability.

Target Range: E.g., Less than 10 fault reports per month.

Maintainability and Related Qualities:

- **Maintainability:** How quickly a fault can be corrected once detected.
- **Changeability:** Ease with which software can be modified.
- **Analyzability:** Ease with which causes of failure can be identified, contributing to maintainability. These measurements help quantify and assess the reliability and maintainability of software systems, ensuring they meet desired quality standards.

13.5 SOFTWARE QUALITY MODELS

It is hard to directly measure the quality of a software. However, it can be expressed in terms of several attributes of a software that can be directly measured.

The quality models give a characterization (hierarchical) of software quality in terms of a set of characteristics of the software. The bottom level of the hierarchical can be directly measured, thereby enabling a quantitative assessment of the quality of the software.

There are several well-established quality models.

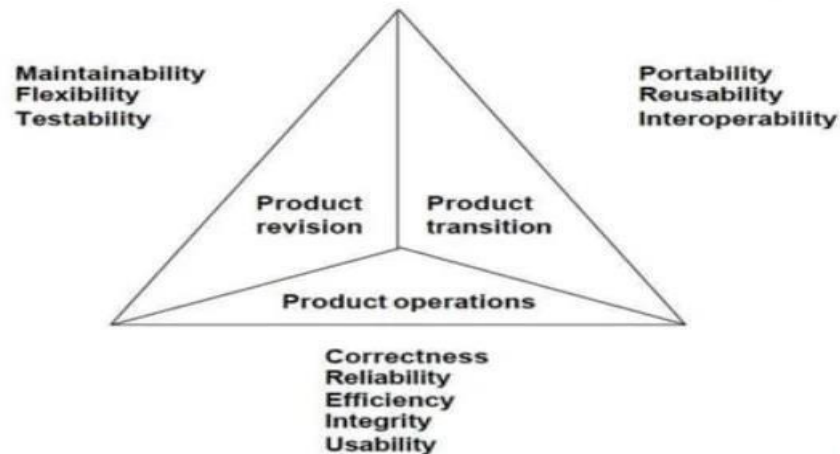
1. McCall's model.
2. Dromey's Model.
3. Boehm's Model.
4. ISO 9126 Model.

Garvin's Quality Dimensions: David Garvin , a professor of Harvard Business school, defined the quality of any product in terms of eight general attributes of the product.

- **Performance:** How well it performs the jobs.
- **Features:** How well it supports the required features.
- **Reliability:** Probability of a product working satisfactorily within a specified period of time.
- **Conformance:** Degree to which the product meets the requirements.
- **Durability:** Measure of the product life.
- **Serviceability:** Speed and effectiveness maintenance.
- **Aesthetics:** The look and feel of the product.
- **Perceived quality:** User's opinion about the product quality.

- 1) **McCall' Model:** McCall defined the quality of a software in terms of three broad parameters: its operational characteristics, how easy it is to fix defects and how easy it is to part it to different platforms. These three high-level quality attributes are defined based on the following eleven attributes of the software:

McCall's Quality Model Triangle



Correctness: The extent to which a software product satisfies its specifications.

Reliability: The probability of the software product working satisfactorily over a given duration.

Efficiency: The amount of computing resources required to perform the required functions.

Integrity: The extent to which the data of the software product remain valid.

Usability: The effort required to operate the software product.

Maintainability: The ease with which it is possible to locate and fix bugs in the software product.

Flexibility: The effort required to adapt the software product to changing requirements.

Testability: The effort required to test a software product to ensure that it performs its intended function.

Portability: The effort required to transfer the software product from one hardware or software system environment to another.

Reusability: The extent to which a software can be reused in other applications.

Interoperability: The effort required to integrate the software with other software.

- 2) **Dromey's model:** Dromey proposed that software product quality depends on four major high-level properties of the software: Correctness, internal characteristics, contextual characteristics and certain descriptive properties. Each of these high-level properties of a software product, in turn depends on several lower-level quality attributes. Dromey's hierarchical quality model is shown in Fig 13.2

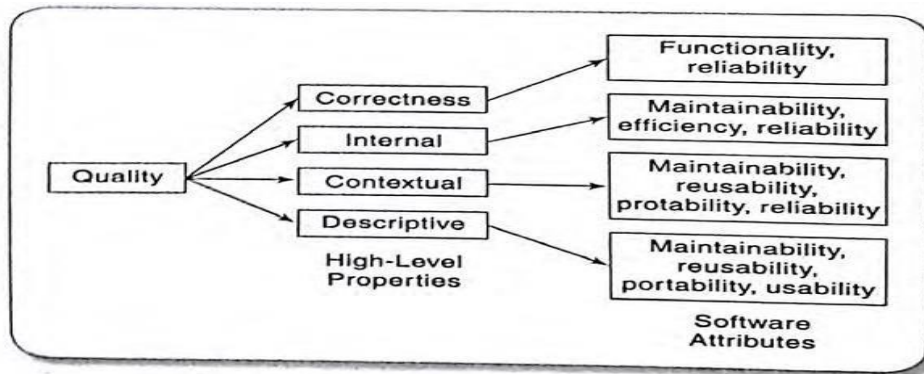


FIGURE 13.2 Dromey's quality model

- 3) **Boehm's Model:** Boehm's suggested that the quality of a software can be defined based on these high-level characteristics that are important for the users of the software. These three high-level characteristics are the following:

As-is -utility: How well (easily, reliably and efficiently) can it be used?

Maintainability: How easy is to understand, modify and then retest the software?

Portability: How difficult would it be to make the software in a changed environment?

Boehm's expressed these high-level product quality attributes in terms of several measurable product attributes. Boehm's hierarchical quality model is shown in Fig 13.3 .

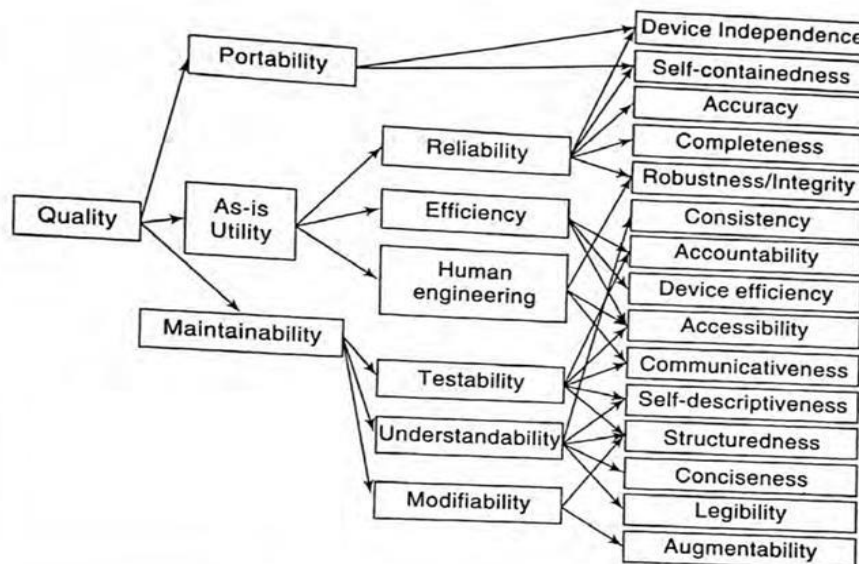


FIGURE 13.3 Boehm's quality model

13.7 PRODUCT AND PROCESS METRICS

Users assess the quality of a software product based on its external attributes, whereas during development, the developers assess the product's quality based on various internal attributes.

The internal attributes may measure either some aspects of product or of the development process (called process metrics).

1. Product Metrics:

Purpose: Measure the characteristics of the software product being developed.

Examples:

Size Metrics: Such as Lines of Code (LOC) and Function Points, which quantify the size or complexity of the software.

Effort Metrics: Like Person-Months (PM), which measure the effort required to develop the software.

Time Metrics: Such as the duration in months or other time units needed to complete the development.

2. Process Metrics:

Purpose: Measure the effectiveness and efficiency of the development process itself.

Examples:

Review Effectiveness: Measures how thorough and effective code reviews are in finding defects.

Defect Metrics: Average number of defects found per hour of inspection, average time taken to correct defects, and average number of failures detected during testing per line of code.

Productivity Metrics: Measures the efficiency of the development team in terms of output per unit of effort or time.

Quality Metrics: Such as the number of latent defects per line of code, which indicates the robustness of the software after development.

Differences:

- **Focus:** Product metrics focus on the characteristics of the software being built (size, effort, time), while process metrics focus on how well the development process is performing (effectiveness, efficiency, quality).
- **Use:** Product metrics are used to gauge the attributes of the final software product, aiding in planning, estimation, and evaluation. Process metrics help in assessing and improving the development process itself, aiming to enhance quality, efficiency, and productivity.
- **Application:** Product metrics are typically applied during and after development phases to assess the product's progress and quality. Process metrics are applied throughout the development lifecycle to monitor and improve the development process continuously.

By employing both types of metrics effectively, software development teams can better manage projects, optimize processes, and deliver high-quality software products that meet user expectations.

13.8 PRODUCT VERSUS PROCESS QUALITY MANAGEMENT

In software development, managing quality can be approached from two main perspectives: product quality management and process quality management. Here's a breakdown of each approach and their key aspects:

Product Quality Management

Product quality management focuses on evaluating and ensuring the quality of the software product itself. This approach is typically more straightforward to implement and measure after the software has been developed.

Aspects:

1. Measurement Focus: Emphasizes metrics that assess the characteristics and attributes of the final software product, such as size (LOC, function points), reliability (defects found per LOC), performance (response time), and usability (user satisfaction ratings).

2. Evaluation Timing: Product quality metrics are often measured and evaluated after the software product has been completed or at significant milestones during development.

3. Benefits:

- Provides clear benchmarks for evaluating the success of the software development project.
- Facilitates comparisons with user requirements and industry standards.
- Helps in identifying areas for improvement in subsequent software versions or projects.

4. Challenges:

- Predicting final product quality based on intermediate stages (like early code modules or prototypes) can be challenging.
- Metrics may not always capture the full complexity or performance of the final integrated product.

Process Quality Management

Process quality management focuses on assessing and improving the quality of the development processes used to create the software. This approach aims to reduce errors and improve efficiency throughout the development lifecycle.

Aspects:

1. Measurement Focus: Emphasizes metrics related to the development processes themselves, such as defect detection rates during inspections, rework effort, productivity (e.g., lines of code produced per hour), and adherence to defined standards and procedures.

2. Evaluation Timing: Process quality metrics are monitored continuously throughout the development lifecycle, from initial planning through to deployment and maintenance.

3. Benefits:

- Helps in identifying and correcting errors early in the development process, reducing the cost and effort of rework.
- Facilitates continuous improvement of development practices, leading to higher overall quality in software products.
- Provides insights into the effectiveness of development methodologies and practices used by the team.

4. Challenges:

- Requires consistent monitoring and analysis of metrics throughout the development lifecycle.
- Effectiveness of process improvements may not always translate directly into improved product quality without careful management and integration.

Integration and Synergy

- While product and process quality management approaches have distinct focuses, they are complementary.
- Effective software development teams often integrate both approaches to achieve optimal results.

SOFTWARE PROJECT ESTIMATION

Software cost and effort estimation will never be an exact science. Too many variables—human, technical, environmental, political—can affect the ultimate cost of software and effort applied to develop it. However, software project estimation can be transformed from a black art to a series of systematic steps that provide estimates with acceptable risk. To achieve reliable cost and effort estimates, a number of options arise:

1. Delay estimation until late in the project (obviously, we can achieve 100 per cent accurate estimates after the project is complete!).
2. Base estimates on similar projects that have already been completed.
3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
4. Use one or more empirical models for software cost and effort estimation.

Unfortunately, the first option, however attractive, is not practical. Cost estimates must be provided upfront. However, you should recognize that the longer you wait, the more you know, and the more you know, the less likely you are to make serious errors in your estimates.

The second option can work reasonably well, if the current project is quite similar to past efforts and other project influences (e.g., the customer, business conditions, the software engineering environment, deadlines) are roughly equivalent. Unfortunately, past experience has not always been a good indicator of future results.

The remaining options are viable approaches to software project estimation. Ideally, the techniques noted for each option should be applied in tandem; each used as a cross-check for the other. Decomposition techniques take a divide-and-conquer approach to software project estimation. By decomposing a project into major functions and related software engineering activities, cost and effort estimation can be performed in a stepwise fashion. Empirical estimation models can be used to complement decomposition techniques and offer a potentially valuable estimation approach in their own right. A model is based on experience (historical data) and takes the form

$$d = f(v_i)$$

where d is one of a number of estimated values (e.g., effort, cost, project duration) and v_i are selected independent parameters (e.g., estimated LOC or FP).

Automated estimation tools implement one or more decomposition techniques or empirical models and provide an attractive option for estimating. In such systems, the characteristics of the development organization (e.g., experience, environment) and the software to be developed are described. Cost and effort estimates are derived from these data.

Each of the viable software cost estimation options is only as good as the historical data used to seed the estimate. If no historical data exist, costing rests on a very shaky foundation. In Chapter 25, we examined the characteristics of some of the software metrics that provide the basis for historical estimation data.

DECOMPOSITION TECHNIQUES

Software project estimation is a form of problem solving, and in most cases, the problem to be solved (i.e., developing a cost and effort estimate for a software project) is too complex to be considered in one piece. For this reason, you should decompose the problem, recharacterizing it as a set of smaller (and hopefully, more manageable) problems.

In Chapter 24, the decomposition approach was discussed from two different points of view: decomposition of the problem and decomposition of the process. Estimation uses one or both forms of partitioning. But before an estimate can be made, you must understand the scope of the software to be built and generate an estimate of its “size.”

Software Sizing

The accuracy of a software project estimate is predicated on a number of things:

- (1) the degree to which you have properly estimated the size of the product to be built;
- (2) the ability to translate the size estimate into human effort, calendar time, and dollars (a function of the availability of reliable software metrics from past projects);
- (3) the degree to which the project plan reflects the abilities of the software team; and
- (4) the stability of product requirements and the environment that supports the software engineering effort.

In this section, I consider the software sizing problem. Because a project estimate is only as good as the estimate of the size of the work to be accomplished, sizing represents your first major challenge as a planner. In the context of project planning, size refers to a quantifiable outcome of the software project. If a direct approach is taken, size can be measured in lines of code (LOC). If an indirect approach is chosen, size is represented as function points (FP).

Putnam and Myers [Put92] suggest four different approaches to the sizing problem:

- “Fuzzy logic” sizing. This approach uses the approximate reasoning techniques that are the cornerstone of fuzzy logic. To apply this approach, the planner must identify the type of application, establish its magnitude on a qualitative scale, and then refine the magnitude within the original range.
- Function point sizing. The planner develops estimates of the information domain characteristics discussed in Chapter 23.
- Standard component sizing. Software is composed of a number of different “standard components” that are generic to a particular application area. For example, the standard components for an information system are subsystems, modules, screens, reports, interactive programs, batch programs, files, LOC, and object-level instructions. The project planner estimates the number of occurrences of each standard component and then uses historical project data to estimate the delivered size per standard component.
- Change sizing. This approach is used when a project encompasses the use of existing software that must be modified in some way as part of a project. The planner estimates the number and type (e.g., reuse, adding code, changing code, deleting code) of modifications that must be accomplished.

Putnam and Myers suggest that the results of each of these sizing approaches be combined statistically to create a three-point or expected-value estimate. This is accomplished by developing optimistic (low), most likely, and pessimistic (high) values for size and combining them using Equation , described in Section .

Problem-Based Estimation

In Chapter 25, lines of code and function points were described as measures from which productivity metrics can be computed. LOC and FP data are used in two ways during software project estimation: (1) as estimation variables to “size” each element of the software and (2) as baseline metrics collected from past projects and used in conjunction with estimation variables to develop cost and effort projections.

LOC and FP estimation are distinct estimation techniques. Yet both have a number of characteristics in common. You begin with a bounded statement of software scope and from this statement attempt to decompose the statement of scope into problem functions that can each be estimated individually. LOC or FP (the estimation variable) is then estimated for each function. Alternatively, you may choose another component for sizing, such as classes or objects, changes, or business processes affected.

Baseline productivity metrics (e.g., LOC/pm or FP/pm⁶) are then applied to the appropriate estimation variable, and cost or effort for the function is derived. Function estimates are combined to produce an overall estimate for the entire project.

It is important to note, however, that there is often substantial scatter in productivity metrics for an organization, making the use of a single-baseline productivity metric suspect. In general, LOC/pm or FP/pm averages should be computed by project domain. That is, projects should be grouped by team size, application area, complexity, and other relevant parameters. Local domain averages should then be computed. When a new project is estimated, it should first be allocated to a domain, and then the appropriate domain average for past productivity should be used in generating the estimate.

The LOC and FP estimation techniques differ in the level of detail required for decomposition and the target of the partitioning. When LOC is used as the estimation variable, decomposition is absolutely essential and is often taken to considerable levels of detail. The greater the degree of partitioning, the more likely reasonably accurate estimates of LOC can be developed.

For FP estimates, decomposition works differently. Rather than focusing on function, each of the information domain characteristics—inputs, outputs, data files, inquiries, and external interfaces—as well as the 14 complexity adjustment values discussed in Chapter 23 are estimated. The resultant estimates can then be used to derive an FP value that can be tied to past data and used to generate an estimate.

Regardless of the estimation variable that is used, you should begin by estimating a range of values for each function or information domain value. Using historical data or (when all else fails) intuition, estimate an optimistic, most likely, and pessimistic size value for each function or count for each information domain value. An implicit indication of the degree of uncertainty is provided when a range of values is specified.

A three-point or expected value can then be computed. The expected value for the estimation variable (size) S can be computed as a weighted average of the optimistic (s_{opt}), most likely (s_m), and pessimistic (s_{pess}) estimates. For example,

$$S = \frac{s_{opt} + 4s_m + s_{pess}}{6}$$

gives heaviest credence to the “most likely” estimate and follows a beta probability distribution. We assume that there is a very small probability the actual size result will fall outside the optimistic or pessimistic values.

Once the expected value for the estimation variable has been determined, historical LOC or FP productivity data are applied. Are the estimates correct? The only reasonable answer to this question is: “You can’t be sure.” Any estimation technique, no matter how sophisticated, must be cross-checked with another approach. Even then, common sense and experience must prevail.

An Example of LOC-Based Estimation

As an example of LOC and FP problem-based estimation techniques, I consider a software package to be developed for a computer-aided design application for mechanical components. The software is to execute on an engineering workstation and must interface with various computer graphics peripherals including a mouse, digitizer, high-resolution color display, and laser printer. A preliminary statement of software scope can be developed:

The mechanical CAD software will accept two- and three-dimensional geometric data from an engineer. The engineer will interact and control the CAD system through a user interface that will exhibit characteristics of good human/machine interface design. All geometric data and other supporting information will be maintained in a CAD database. Design analysis modules will be developed to produce the required output, which will be displayed on a variety of graphics devices. The software will be designed to control and interact with peripheral devices that include a mouse, digitizer, laser printer, and plotter.

This statement of scope is preliminary—it is not bounded. Every sentence would have to be expanded to provide concrete detail and quantitative bounding. For example, before estimation can begin, the planner must determine what “characteristics of good human/machine interface design” means or what the size and sophistication of the “CAD database” are to be. For our purposes, assume that further refinement has occurred and that the major software functions listed in Figure are identified. Following the decomposition technique for LOC, an estimation table (Figure) is developed. A range of LOC estimates is developed for each function. For example, the range of LOC estimates for the 3D geometric analysis function is optimistic, 4600 LOC; most likely, 6900 LOC; and pessimistic, 8600 LOC. Applying Equation 26.1, the expected value for the 3D geometric analysis function is 6800 LOC. Other estimates are derived in a similar

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
<i>Estimated lines of code</i>	<i>33,200</i>

fashion. By summing vertically in the estimated LOC column, an estimate of 33,200 lines of code is established for the CAD system.

A review of historical data indicates that the organizational average productivity for systems of this type is 620 LOC/pm. Based on a burdened labor rate of \$8000 per month, the cost per line of code is approximately \$13. Based on the LOC estimate and the historical productivity data, the total estimated project cost is \$431,000 and the estimated effort is 54 person-months.

An Example of FP-Based Estimation

Decomposition for FP-based estimation focuses on information domain values rather than software functions. Referring to the table presented in Figure, you would estimate inputs, outputs, inquiries, files, and external interfaces for the CAD software. An FP value is computed using the technique discussed in Chapter 23. For the purposes of this estimate, the complexity weighting factor is assumed to be average. Figure presents the results of this estimate.

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
<i>Count total</i>						<i>320</i>

Each of the complexity weighting factors is estimated, and the value adjustment factor is computed as described in Chapter 23:

Factor	Value
Backup and recovery	4
Data communications	2
Distributed processing	0
Performance critical	4
Existing operating environment	3
Online data entry	4
Input transaction over multiple screens	5
Master files updated online	3
Information domain values complex	5
Internal processing complex	5
Code designed for reuse	4
Conversion/installation in design	3
Multiple installations	5
Application designed for change	5
Value adjustment factor	1.17

Finally, the estimated number of FP is derived:

$$FP_{estimated} = \text{count total} \times [0.65 + 0.01 + \sum(F_i)] = 375$$

The organizational average productivity for systems of this type is 6.5 FP/pm. Based on a burdened labor rate of \$8000 per month, the cost per FP is approximately \$1230. Based on the FP estimate and the historical productivity data, the total estimated project cost is \$461,000 and the estimated effort is 58 person-months.

Process-Based Estimation

The most common technique for estimating a project is to base the estimate on the process that will be used. That is, the process is decomposed into a relatively small set of tasks and the effort required to accomplish each task is estimated.

Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
Task →				Analysis	Design	Code	Test		
Function									
Y									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
Totals	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
% effort	1%	1%	1%	8%	45%	10%	36%		

CC = customer communication CE = customer evaluation

Like the problem-based techniques, process-based estimation begins with a delineation of software functions obtained from the project scope. A series of frame work activities must be performed for each function. Functions and related frame work activities⁸ may be represented as part of a table similar to the one presented in Figure.

Once problem functions and process activities are melded, you estimate the effort (e.g., person-months) that will be required to accomplish each software process activity for each software function. These data constitute the central matrix of the table in Figure. Average labor rates (i.e., cost/unit effort) are then applied to the effort estimated for each process activity. It is very likely the labor rate will vary for each task. Senior staff are heavily involved in early framework activities and are gen erally more expensive than junior staff involved in construction and release.

Costs and effort for each function and framework activity are computed as the last step. If process-based estimation is performed independently of LOC or FP estima tion, we now have two or three estimates for cost and effort that may be compared and reconciled. If both sets of estimates show reasonable agreement, there is good reason to believe that the estimates are reliable. If, on the other hand, the results of these decomposition techniques show little agreement, further investigation and analysis must be conducted.

An Example of Process-Based Estimation

To illustrate the use of process-based estimation, consider the CAD software intro duced in Section. The system configuration and all software functions remain unchanged and are indicated by project scope.

Referring to the completed process-based table shown in Figure, estimates of effort (in person-months) for each software engineering activity are provided for each CAD software function (abbreviated for brevity). The engineering and con struction release activities are subdivided into the major software engineering tasks shown. Gross estimates of effort are provided for customer communication, plan ning, and risk analysis. These are noted in the total row at the bottom of the table. Horizontal and vertical totals provide an indication of estimated effort required for analysis, design, code, and test. It should be noted that 53 percent of all effort is ex pended on front-end engineering tasks (requirements analysis and design), indicat ing the relative importance of this work.

Based on an average burdened labor rate of \$8000 per month, the total estimated project cost is \$368,000 and the estimated effort is 46 person-months. If desired, labor rates could be associated with each framework activity or software engineering task and computed separately.

Estimation with Use Cases

As I have noted throughout Part 2 of this book, use cases provide a software team with insight into software scope and requirements. However, developing an estimation approach with use cases is problematic for the following reasons [Smi99]:

- Use cases are described using many different formats and styles—there is no standard form.
- Use cases represent an external view (the user’s view) of the software and can therefore be written at many different levels of abstraction.
- Use cases do not address the complexity of the functions and features that are described.
- Use cases can describe complex behavior (e.g., interactions) that involve many functions and features.

Unlike an LOC or a function point, one person’s “use case” may require months of effort while another person’s use case may be implemented in a day or two.

Although a number of investigators have considered use cases as an estimation input, no proven estimation method has emerged to date.⁹ Smith [Smi99] suggests that use cases can be used for estimation, but only if they are considered within the context of the “structural hierarchy” that they are used to describe.

Smith argues that any level of this structural hierarchy can be described by no more than 10 use cases. Each of these use cases would encompass no more than 30 distinct scenarios. Obviously, use cases that describe a large system are written at a much higher level of abstraction (and represent considerably more development effort) than use cases that are written to describe a single subsystem. Therefore, before use cases can be used for estimation, the level within the structural hierarchy is established, the average length (in pages) of each use case is determined, the type of software (e.g., real-time, business, engineering/scientific, WebApp, embedded) is defined, and a rough architecture for the system is considered. Once these characteristics are established, empirical data may be used to establish the estimated number of LOC or FP per use case (for each level of the hierarchy). Historical data are then used to compute the effort required to develop the system.

To illustrate how this computation might be made, consider the following relationship:

$$\text{LOC estimate} = N \times \text{LOC}_{avg} + [(S_a / S_h - 1) + (P_a / P_h - 1)] \times \text{LOC}_{adjust}$$

Where

N = actual number of use cases (26.2)

LOC_{avg} = historical average LOC per use case for this type of subsystem

LOC_{adjust} = represents an adjustment based on n percent of LOC_{avg} where n is defined locally and represents the difference between this project and “average” projects

S_a = actual scenarios per use case

S_h = average scenarios per use case for this type of subsystem

P_a = actual pages per use case

P_h = average pages per use case for this type of subsystem

Expression could be used to develop a rough estimate of the number of LOC based on the actual number of use cases adjusted by the number of scenarios and the page length of the use cases. The adjustment represents up to n percent of the historical average LOC per use case.

An Example of Use-Case–Based Estimation

The CAD software introduced in Section is composed of three subsystem groups: user interface subsystem (includes UICF), engineering subsystem group (includes the 2DGA, 3DGA, and DAM subsystems), and infrastructure subsystem group (includes CGDF and PCF subsystems). Six use cases describe the user interface subsystem. Each use case is described by no more than 10 scenarios and has an average length of six pages. The engineering subsystem group is described by 10 use cases (these are considered to be at a higher level of the structural hierarchy). Each of these use cases has no more than 20 scenarios associated with it and has an average length of eight pages. Finally, the infrastructure subsystem group is described by five use cases with an average of only six scenarios and an average length of five pages.

	use cases	scenarios	pages	scenarios	pages	LOC	LOC estimate
User interface subsystem	6	10	6	12	5	560	3,366
Engineering subsystem group	10	20	8	16	8	3100	31,233
Infrastructure subsystem group	5	6	5	10	6	1650	7,970
Total LOC estimate							42,568

Using the relationship noted in Expression (26.2) with $n = 30$, the table shown in Figure is developed. Considering the first row of the table, historical data indicate that UI software requires an average of 800 LOC per use case when the use case has no more than 12 scenarios and is described in less than five pages. These data conform reasonably well for the CAD system. Hence the LOC estimate for the user interface subsystem is computed using expression. Using the same approach, estimates are made for both the engineering and infrastructure subsystem groups. Figure summarizes the estimates and indicates that the overall size of the CAD is estimated at 42,500 LOC.

Using 620 LOC/pm as the average productivity for systems of this type and aburdened labor rate of \$8000 per month, the cost per line of code is approxi mately \$13. Based on the use-case estimate and the historical productivity data, the total estimated project cost is \$552,000 and the estimated effort is 68 person months.

Reconciling Estimates

The estimation techniques discussed in the preceding sections result in multiple estimates that must be reconciled to produce a single estimate of effort, project duration, or cost. To illustrate this reconciliation procedure, I again consider the CAD software introduced in Section .

The total estimated effort for the CAD software ranges from a low of 46 person months (derived using a process-based estimation approach) to a high of 68 person months (derived with use-case estimation). The

average estimate (using all four approaches) is 56 person-months. The variation from the average estimate is approximately 18 percent on the low side and 21 percent on the high side.

What happens when agreement between estimates is poor? The answer to this question requires a reevaluation of information used to make the estimates. Widely divergent estimates can often be traced to one of two causes: (1) the scope of the project is not adequately understood or has been misinterpreted by the planner, or (2) productivity data used for problem-based estimation techniques is inappropriate for the application, obsolete (in that it no longer accurately reflects the software engineering organization), or has been misapplied. You should determine the cause of divergence and then reconcile the estimates.

EMPIRICAL ESTIMATION MODELS

An estimation model for computer software uses empirically derived formulas to predict effort as a function of LOC or FP.¹¹ Values for LOC or FP are estimated using the approach described in Sections and. But instead of using the tables described in those sections, the resultant values for LOC or FP are plugged into the estimation model.

The empirical data that support most estimation models are derived from a limited sample of projects. For this reason, no estimation model is appropriate for all classes of software and in all development environments. Therefore, you should use the results obtained from such models judiciously.

An estimation model should be calibrated to reflect local conditions. The model should be tested by applying data collected from completed projects, plugging the data into the model, and then comparing actual to predicted results. If agreement is poor, the model must be tuned and retested before it can be used.

The Structure of Estimation Models

A typical estimation model is derived using regression analysis on data collected from past software projects. The overall structure of such models takes the form [Mat94]

$$E = A + B \times (e_v)^C$$

where A, B, and C are empirically derived constants, E is effort in person-months, and e_v is the estimation variable (either LOC or FP). In addition to the relationship noted in Equation , the majority of estimation models have some form of project adjustment component that enables E to be adjusted by other project characteristics (e.g., problem complexity, staff experience, development environment). Among the many LOC-oriented estimation models proposed in the literature are

$$E = 5.2 \times (\text{KLOC})^{0.91} \quad \text{Walston-Felix model}$$

$$E = 5.5 \cdot 0.73 (\text{KLOC})^{1.16} \quad \text{Bailey-Basili model}$$

$$E = 3.2 (\text{KLOC})^{1.05} \quad \text{Boehm simple model}$$

$$E = 5.288 ((\text{KLOC})^{0.91}) \quad \text{Doty model for KLOC} > 9$$

FP-oriented models have also been proposed. These include

$$E = -91.4 + 0.355 \text{ FP} \quad \text{Albrecht and Gaffney model}$$

$$E = -37 + 0.96 \text{ FP}$$

Kemerer model

$$E = 12.88 + 0.405 \text{ FP}$$

Small project regression model

A quick examination of these models indicates that each will yield a different result for the same values of LOC or FP. The implication is clear. Estimation models must be calibrated for local needs!

The COCOMO II Model

In his classic book on “software engineering economics,” Barry Boehm [Boe81] introduced a hierarchy of software estimation models bearing the name COCOMO, for CONstructive COst MOdel. The original COCOMO model became one of the most widely used and discussed software cost estimation models in the industry. It has evolved into a more comprehensive estimation model, called COCOMOII[Boe00]. Like its predecessor, COCOMOII is actually a hierarchy of estimation models that address the following areas:

- Application composition model. Used during the early stages of software engineering, when prototyping of user interfaces, consideration of software and system interaction, assessment of performance, and evaluation of technology maturity are paramount.
- Early design stage model. Used once requirements have been stabilized and basic software architecture has been established.
- Post-architecture-stage model. Used during the construction of the software.

Like all estimation models for software, the COCOMO II models require sizing information. Three different sizing options are available as part of the model hierarchy: object points, function points, and lines of source code.

Object type	Complexity weight		
	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
3GL component			10

The COCOMO II application composition model uses object points and is illustrated in the following paragraphs. It should be noted that other, more sophisticated estimation models (using FP and KLOC) are also available as part of COCOMO II.

Like function points, the object point is an indirect software measure that is computed using counts of the number of (1) screens (at the user interface), (2) reports, and (3) components likely to be required to build the application. Each object instance (e.g., a screen or report) is classified into one of three complexity levels (i.e., simple, medium, or difficult) using criteria suggested by Boehm [Boe96]. In essence, complexity is a function of the number and source of the client and server data tables that are required to generate the screen or report and the number of views or sections presented as part of the screen or report.

Once complexity is determined, the number of screens, reports, and components are weighted according to the table illustrated in Figure. The object point count is then determined by multiplying the original

number of object instances by the weighting factor in the figure and summing to obtain a total object point count. When component-based development or general software reuse is to be applied, the percent of reuse (%reuse) is estimated and the object point count is adjusted:

$$NOP = (\text{object points}) \times [(100 - \%reuse)/100]$$

where NOP is defined as new object points.

To derive an estimate of effort based on the computed NOP value, a “productivity rate” must be derived. Figure 26.7 presents the productivity rate

$$PROD = \frac{NOP}{\text{person-month}}$$

for different levels of developer experience and development environment maturity.

Once the productivity rate has been determined, an estimate of project effort is computed using

$$\text{estimated effort} = \frac{NOP}{PROD}$$

In more advanced COCOMO II models,12 a variety of scale factors, cost drivers, and adjustment procedures are required. A complete discussion of these is beyond

Developer's experience/capability	Very low	Low	Nominal	High	Very high
Environment maturity/capability	Very low	Low	Nominal	High	Very high
PROD	4	7	13	25	50

the scope of this book.If you have further interest, see [Boe00] or visit the COCOMOII website.

The Software Equation

The software equation [Put92] is a dynamic multivariable model that assumes a spe cific distribution of effort over the life of a software development project. The model has been derived from productivity data collected for over 4000 contemporary soft ware projects. Based on these data, we derive an estimation model of the form

$$E = \frac{LOC \times B^{0.333}}{p^3} \times \frac{1}{t^4}$$

Where

E = effort in person-months or person-years

t = project duration in months or years

B = “special skills factor”

P = “productivity parameter” that reflects: overall process maturity and man agement practices, the extent to which good software engineering practices are used, the level of programming languages used,

the state of the software environment, the skills and experience of the software team, and the complexity of the application

Typical values might be $P = 2000$ for development of real-time embedded software, $P = 10,000$ for telecommunication and systems software, and $P = 28,000$ for business systems applications. The productivity parameter can be derived for local conditions using historical data collected from past development efforts.

You should note that the software equation has two independent parameters: (1) an estimate of size (in LOC) and (2) an indication of project duration in calendar months or years.

To simplify the estimation process and use a more common form for their estimation model, Putnam and Myers [Put92] suggest a set of equations derived from the software equation. Minimum development time is defined as

$$t_{min} = 8.14 \frac{LOC}{p^{0.43}} \text{ in months for } t_{min} > 6 \text{ months}$$

$$E = 180 B t^3 \text{ in person-months for } E \geq 20 \text{ person-months}$$

Note that t in Equation is represented in years.

Using Equation with $P = 12,000$ (the recommended value for scientific software) for the CAD software discussed earlier in this chapter,

$$t_{min} = 8.14 \times \frac{33,200}{12000^{0.43}} = 12.6 \text{ calendar months}$$

$$E = 180 \times 0.28 \times (1.05)^3 = 58 \text{ person-months}$$

The results of the software equation correspond favorably with the estimates developed in Section . Like the COCOMO model noted in Section , the software equation continues to evolve.